BharatBridge (v4.2) — Technical & Architectural Blueprint

VERSION 4.2

Real-Time Multilingual Conference Intelligence • Developer Handover Specification

> **Executive Summary**
> BharatBridge is a real-time multilingual event platform enabling instant live speech-to-text (STT), Gemini LLM context-aware neural translation, and sequential text-to-speech (TTS) synthesis across 22 official Indian languages + English.

1. Key Technical Features & Value Proposition

- **Context-Aware Natural LLM Translation:** Powered by Google Gemini AI (Gemini 2.5/2.0 Flash) with conversational context-window prompts. Never translates word-for-word literally; preserves natural grammar, idioms, and conversational tone.
- **Strict Speaker vs. Listener Role Isolation:** Speaker screen displays ONLY original spoken speech captions. Target translations and target audio TTS are suppressed for speakers to prevent confusion. Listeners receive real-time target translations and audio TTS.
- **Non-Blocking Sentence Queue & STT Deduplication:** Web Speech API inputs pass through a FIFO queue (`sentenceQueueRef`) with a 2.5-second deduplication lock window to prevent line bleeding or duplicate STT invocations. Speech recognition runs continuously in parallel.
- **Zero-Overlapping Sequential TTS Audio Queue:** Audio clips are managed by a single active player (`activeAudioElRef`). Incoming translated lines play strictly back-to-back in sequential order (`ttsQueueRef`) with zero voice overlapping.
- **Instant Guest Mode & Sub-100ms Catch-Up:** 100% Guest Mode compatibility without mandatory logins. Newly joined listeners automatically receive the entire translated session history upon entering room PIN.
- **Complete Analytics & Export Options:** Download transcripts as TXT, CSV (with UTF-8 BOM `\uFEFF` for native Excel Devanagari rendering), WAV audio recordings, and Gemini-generated AI Session Summaries.

2. System Architecture & Data Flow

Component	Technology Stack	Function & Architectural Guarantee
Real-Time Gateway	Node.js `ws` + Express	Sub-100ms WebSocket subscription. Non-blocking room creation (`<1ms`).
Contextual AI Engine	Gemini 2.5 Flash / 2.0 Flash	Context-aware translation pipeline using recent 4–6 sentence pairs.
STT Engine	Web Speech API + PCM 16kHz Stream	Continuous client speech-to-text with 2.5s deduplication lock.
TTS Neural Engine	Azure Speech / Google Neural TTS	Sequential audio synthesis across 22 Indian language neural voices.
Storage & Exports	Azure Blob Storage + Firebase	Dual-format JSON/CSV logging and WAV audio exports.
DNS Guard	`dns.setDefaultResultOrder("ipv4first")`	Fixes Windows IPv6 connection timeouts on Node fetch.

3. API Endpoints Specification

Endpoint	Method	Description
`/api/health`	GET	Checks system status, execution mode, and Azure storage connectivity.
`/api/translate`	POST	Context-aware translation API powered by Gemini AI.
`/api/session-summary`	POST	Generates structured AI summary (Overview, Key Points, Notes) from session transcript.
`/api/azure/tts`	POST	Synthesizes neural speech MP3 audio using Azure Speech / Google TTS.

Endpoint	Method	Description
`/api/azure/upload-caption`	POST	Uploads session captions to Azure Blob Storage in CSV + JSON format.

4. WebSocket Message Schemas

```
1. Join Session (Client -> Server):
   { "type": "join", "sessionId": "582941", "role": "listener", "targetLang": "mr" }

2. Spoken Text Translation (Speaker -> Server):
   { "type": "text_translation", "originalText": "Hello", "sourceLang": "en", "targetLang": "mr", "isFinal": true }

3. Live Transcription Stream (Server -> Listener):
   { "type": "transcription", "originalText": "Hello", "translatedText": "नमस्कार", "targetLang": "mr" }

4. Session History Broadcast (Server -> Listener):
   { "type": "history", "messages": [ { "originalText": "...", "translatedText": "..." } ] }
```

5. Master System Prompt Guidelines

> **System Prompt Configuration for Developers / AI Agents**
>
> Always maintain: (1) Instant non-blocking WebSockets (<1ms), (2) Gemini context-aware translation, (3) Speaker vs Listener role isolation, (4) Non-blocking sentence queueing with 2.5s STT deduplication lock, (5) Zero-overlap sequential TTS audio playback, (6) Clean 2-column UI layout.

6. Launch & Deployment Guide

```
# Install dependencies
npm install

# TypeScript Type Check
npx tsc --noEmit

# Start Production/Development Server
npx tsx server.ts

# Optional Public Localtunnel
npx localtunnel --port 3000
```